KARPAGAM
COLLEGE OF ENGINEERING
Rediscover | Refine | Redefine
Accredited by NAAC with 'A+' grade
Autonomous | Affiliated to Anna University
(An ISO 9001:2015 and ISO 14001:2015 Certified Institution)

**ACADEMIC YEAR: 2026-27**

**ASSIGNMENT FRONT SHEET**

| | |
|---|---|
| **Name of the Student:** | **SAMUEL N** |
| **Register Number:** | **717825P342** |
| **Course Code & Title:** | **23CSR306 JAVA PROGRAMMING** |
| **Year / Dept. / Sem / Section:** | **II / CSE/ III /C** |
| **Assignment:** | **II – OOP Mini Project (any one of 35)** |
| **Specification No. & Title:** | **3 - Online Course Enrollment System** |
| **GitHub Repository Link:** | **https://github.com/Samkarpagam07/Java-Assignment/upload/main** |
| **Date of Issue:** | |
| **Date of Submission:** | |

**Marks Details**

| CO / K Level | Maximum Marks | Marks Obtained |
|---|---|---|
| **CO4 / K3** | 50 | |
| **CO5 / K3** | 50 | |
| **Total** | **100** | |

**Course In-charge**

KARPAGAM COLLEGE OF ENGINEERING
Rediscover | Refine | Redefine
Accredited by NAAC with A+ grade
Autonomous | Affiliated to Anna University
(An ISO 9001:2015 and ISO 14001:2015 Certified Institution)

**Rubrics for Assignment Evaluation**

| | | | | | |
|---|---|---|---|---|---|
| **Assignment Number:** | : | II | **Date** | : | |
| **Course Code & Title** | : | 23CSR306 Java Programming | **Specification No. & Title** | : | 3 - Online Course Enrollment System |
| **Register Number** | : | 717825P342 | **Name of the Student** | : | SAMUEL N |
| **Name of the Evaluator** | : | | **GitHub Link** | : | https://github.com/Samkarpagam07/Java-Assignment/upload/main |

*Marks for each criterion = Level (1 – 5) × Weightage. Total: 100 (CO4: 50 + CO5: 50).*

| Criterion | CO | Wt. | Exemplary (5) | Excellent (4) | Adequate (3) | Average (2) | Inadequate (1) | Max | Marks |
|---|---|---|---|---|---|---|---|---|---|
| **Program Correctness & Business Rules** | CO4 | 4 | All menu options work; every business rule enforced correctly. | All options work; minor gaps in one rule. | Main flow works; some rules missing. | Partially working; several rules not handled. | Does not run or most features missing. | 20 | |
| **Input Validation & Exception Handling** | CO4 | 3 | All inputs validated; custom exceptions; program never crashes. | Most inputs validated with try-catch. | Basic validation; some invalid inputs not handled. | Little validation; crashes on bad input. | No validation or exception handling. | 15 | |
| **Menu-driven Interface & Expected Outputs** | CO4 | 3 | Clear menu; all expected outputs (bills/receipts/status) shown neatly. | Clear menu; most expected outputs shown. | Menu works; outputs basic or incomplete. | Menu unclear; few outputs shown. | No proper menu or outputs. | 15 | |
| **OOP Design (Encapsulation, Inheritance, Aggregation, Polymorphism)** | CO5 | 4 | All four concepts applied meaningfully; all required classes present. | Concepts applied well; minor design gaps. | Classes present; concepts used in a basic way. | Weak design; some required classes missing. | No OOP design; mostly procedural. | 20 | |
| **Code Quality & Package Structure** | CO5 | 2 | Clean packages, naming, indentation and comments. | Good structure with few lapses. | Acceptable structure; average readability. | Poor naming / structure. | Single file; unreadable code. | 10 | |
| **GitHub Repository & README** | CO5 | 1 | Complete code with clear README (problem, run steps, name, roll no.). | Complete code; README missing minor details. | Code present; README incomplete. | Partial code; no README. | No repository. | 5 | |
| **Viva / Explanation** | CO5 | 3 | Explains design, rules and code clearly; answers all questions. | Explains well; answers most questions. | Explains main logic; answers some questions. | Limited explanation of own code. | Unable to explain the code. | 15 | |
| | | | | | | | **CO4 Total (Criteria 1 – 3)** | **50** | |
| | | | | | | | **CO5 Total (Criteria 4 – 7)** | **50** | |
| | | | | | | | **Total Marks** | **100** | |

**Signature of the Evaluator with date**

## 1. Assignment Question

### Specification 3: Online Course Enrollment System

**Problem Statement:** Design and implement a Java console application for a Course Enrollment System that handles courses, instructors, students, enrollments, and payments.

**Class Requirements:** Course, Instructor, Student, PremiumStudent, Enrollment, Payment, Receipt

**Business Rules:**

**1.** Students can only enroll in available courses.

**2.** Premium students may get fee discounts or access to extra materials.

**3.** Payment must be completed for enrollment confirmation.

**4.** Enrollment should update course capacity.

**5.** Each receipt must link to a student and course.

**Menu Options:** Add Course / Add Student / Enroll in Course / Make Payment / Display Courses / Exit

**Expected Output:**

- Show enrollment confirmation and receipt.
- Update available course capacity.
- Show fee discounts for premium students.

## 2. Java Source Code

**Main.java:**

```
package main;
import java.util.Scanner;
import model.Instructor;
import model.Course;
import model.Student;
import model.PremiumStudent;
import model.Enrollment;
import model.Receipt;
import service.EnrollmentService;
import service.PaymentService;

public class Main {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      // Create instructor
      Instructor instructor = new Instructor(
         "I101", "Arun", "Java Programming"
      );

      // Create course
```

```
Course course = new Course(
    "C101", "Java Fundamentals", 2000.0, 2, instructor
);

// Create services
EnrollmentService enrollmentService = new EnrollmentService();
PaymentService paymentService = new PaymentService();

int studentCount = 0;
int enrollmentCount = 0;
int paymentCount = 0;

Student[] students = new Student[100];
Enrollment[] enrollments = new Enrollment[100];

int choice;

do {
    System.out.println("\n===== ONLINE COURSE ENROLLMENT SYSTEM =====");
    System.out.println("1. Display Course Details");
    System.out.println("2. Register Regular Student");
    System.out.println("3. Register Premium Student");
    System.out.println("4. Enroll Student");
    System.out.println("5. View All Enrollments");
    System.out.println("6. Process Payment");
    System.out.println("7. View All Payments");
    System.out.println("8. Exit");
    System.out.print("Enter your choice: ");

    choice = sc.nextInt();
    sc.nextLine();

    switch (choice) {

        case 1:
            course.displayCourse();
            break;

        case 2:
            if (studentCount >= students.length) {
                System.out.println("Student storage is full!");
                break;
            }

            System.out.print("Enter student ID: ");
            String studentId = sc.nextLine();

            System.out.print("Enter student name: ");
```

```
            String name = sc.nextLine();

            System.out.print("Enter student email: ");
            String email = sc.nextLine();

            students[studentCount] = new Student(
                studentId, name, email
            );

            studentCount++;
            System.out.println("Student registered successfully!");
            break;

        case 3:
            if (studentCount >= students.length) {
                System.out.println("Student storage is full!");
                break;
            }

            System.out.print("Enter student ID: ");
            String premiumId = sc.nextLine();

            System.out.print("Enter student name: ");
            String premiumName = sc.nextLine();

            System.out.print("Enter student email: ");
            String premiumEmail = sc.nextLine();

            System.out.print("Enter discount percentage: ");
            double discount = sc.nextDouble();
            sc.nextLine();

            if (discount < 0 || discount > 100) {
                System.out.println("Invalid discount!");
                break;
            }

            students[studentCount] = new PremiumStudent(
                premiumId, premiumName, premiumEmail, discount
            );

            studentCount++;
            System.out.println("Premium student registered successfully!");
            break;

        case 4:
            if (enrollmentCount >= enrollments.length) {
                System.out.println("Enrollment storage is full!");
```

```
            break;
        }

        if (studentCount == 0) {
            System.out.println("Register a student first!");
            break;
        }

        System.out.println("\nRegistered Students:");

        for (int i = 0; i < studentCount; i++) {
            System.out.println(
                students[i].getStudentId() + " - "
                + students[i].getName()
            );
        }

        System.out.print("Enter student ID to enroll: ");
        String searchId = sc.nextLine();

        Student selectedStudent = null;

        for (int i = 0; i < studentCount; i++) {
            if (students[i].getStudentId().equals(searchId)) {
                selectedStudent = students[i];
                break;
            }
        }

        if (selectedStudent == null) {
            System.out.println("Student not found!");
            break;
        }

        System.out.print("Enter enrollment date (DD-MM-YYYY): ");
        String date = sc.nextLine();

        String enrollmentId = "E" + (enrollmentCount + 101);

        boolean enrolled = enrollmentService.enrollStudent(
            enrollmentId, selectedStudent, course, date
        );

        if (enrolled) {
            enrollments[enrollmentCount] = new Enrollment(
                enrollmentId, selectedStudent, course, date
            );
```

```
                enrollmentCount++;
            }
            break;

        case 5:
            enrollmentService.displayAllEnrollments();
            break;

        case 6:
            if (enrollmentCount == 0) {
                System.out.println("Enroll a student first!");
                break;
            }

            System.out.println("\nEnrollments:");

            for (int i = 0; i < enrollmentCount; i++) {
                System.out.println(
                    enrollments[i].getEnrollmentId() + " - "
                    + enrollments[i].getStudent().getName()
                    + " - " + enrollments[i].getCourse().getCourseName()
                );
            }

            System.out.print("Enter enrollment ID: ");
            String paymentEnrollmentId = sc.nextLine();

            Enrollment selectedEnrollment = null;

            for (int i = 0; i < enrollmentCount; i++) {
                if (enrollments[i].getEnrollmentId()
                        .equals(paymentEnrollmentId)) {
                    selectedEnrollment = enrollments[i];
                    break;
                }
            }

            if (selectedEnrollment == null) {
                System.out.println("Enrollment not found!");
                break;
            }

            System.out.print("Enter payment method: ");
            String method = sc.nextLine();

            double fee = selectedEnrollment.getCourse().getFee();

            if (selectedEnrollment.getStudent() instanceof PremiumStudent) {
```

```
                PremiumStudent premium =
                    (PremiumStudent) selectedEnrollment.getStudent();

                fee = premium.calculateFee(fee);
            }

            String paymentId = "P" + (paymentCount + 101);

            Receipt receipt = paymentService.processPayment(
                paymentId,
                selectedEnrollment,
                fee,
                "24-10-2026",
                method
            );

            if (receipt != null) {
                paymentCount++;

                selectedEnrollment.setStatus("Confirmed");

                receipt.displayReceipt();
            }
            break;

        case 7:
            paymentService.displayAllPayments();
            break;

        case 8:
            System.out.println("Thank you for using the system!");
            break;

        default:
            System.out.println("Invalid choice! Try again.");
      }

    } while (choice != 8);

    sc.close();
  }
}
```

**Course.java:**

```
package model;

public class Course {
  private String courseId;
```

```
    private String courseName;
    private double fee;
    private int capacity;
    private Instructor instructor;

    public Course(String courseId, String courseName, double fee,
                  int capacity, Instructor instructor) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.fee = fee;
        this.capacity = capacity;
        this.instructor = instructor;
    }

    public String getCourseId() {
        return courseId;
    }

    public String getCourseName() {
        return courseName;
    }

    public double getFee() {
        return fee;
    }

    public int getCapacity() {
        return capacity;
    }

    public Instructor getInstructor() {
        return instructor;
    }

    public void setCapacity(int capacity) {
        this.capacity = capacity;
    }

    public void displayCourse() {
        System.out.println("Course ID: " + courseId);
        System.out.println("Course Name: " + courseName);
        System.out.println("Fee: " + fee);
        System.out.println("Capacity: " + capacity);
        System.out.println("Instructor: " + instructor.getName());
    }
}
```

**Instructor.java:**

```
package model;

public class Instructor {
    private String instructorId;
    private String name;
    private String specialization;

    public Instructor(String instructorId, String name, String specialization) {
        this.instructorId = instructorId;
        this.name = name;
        this.specialization = specialization;
    }

    public String getInstructorId() {
        return instructorId;
    }

    public String getName() {
        return name;
    }

    public String getSpecialization() {
        return specialization;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setSpecialization(String specialization) {
        this.specialization = specialization;
    }

    public void displayInstructor() {
        System.out.println("Instructor ID: " + instructorId);
        System.out.println("Name: " + name);
        System.out.println("Specialization: " + specialization);
    }
}
```

**Student.java:**

```
package model;

public class Student {
    private String studentId;
    private String name;
```

```
    private String email;

    public Student(String studentId, String name, String email) {
        this.studentId = studentId;
        this.name = name;
        this.email = email;
    }

    public String getStudentId() {
        return studentId;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void displayStudent() {
        System.out.println("Student ID: " + studentId);
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
    }
}
```

**PremiumStudent.java:**

```
package model;

public class PremiumStudent extends Student {
    private double discount;

    public PremiumStudent(String studentId, String name, String email,
                          double discount) {
        super(studentId, name, email);
        this.discount = discount;
    }

    public double getDiscount() {
```

```
        return discount;
    }

    public void setDiscount(double discount) {
        this.discount = discount;
    }

    public double calculateFee(double originalFee) {
        return originalFee - (originalFee * discount / 100);
    }

    public void displayPremiumStudent() {
        displayStudent();
        System.out.println("Discount: " + discount + "%");
    }
}
```

**Enrollment.java:**

```
package model;

public class Enrollment {
    private String enrollmentId;
    private Student student;
    private Course course;
    private String enrollmentDate;
    private String status;

    public Enrollment(String enrollmentId, Student student, Course course,
                      String enrollmentDate) {
        this.enrollmentId = enrollmentId;
        this.student = student;
        this.course = course;
        this.enrollmentDate = enrollmentDate;
        this.status = "Pending Payment";
    }

    public String getEnrollmentId() {
        return enrollmentId;
    }

    public Student getStudent() {
        return student;
    }

    public Course getCourse() {
        return course;
    }
```

```
    public String getEnrollmentDate() {
        return enrollmentDate;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public void displayEnrollment() {
        System.out.println("Enrollment ID: " + enrollmentId);
        System.out.println("Student: " + student.getName());
        System.out.println("Course: " + course.getCourseName());
        System.out.println("Enrollment Date: " + enrollmentDate);
        System.out.println("Status: " + status);
    }
}
```

**Payment.java:**

```
package model;

public class Payment {
    private String paymentId;
    private String enrollmentId;
    private double amount;
    private String paymentDate;
    private String paymentMethod;
    private String status;

    public Payment(String paymentId, String enrollmentId, double amount,
                   String paymentDate, String paymentMethod) {
        this.paymentId = paymentId;
        this.enrollmentId = enrollmentId;
        this.amount = amount;
        this.paymentDate = paymentDate;
        this.paymentMethod = paymentMethod;
        this.status = "Successful";
    }

    public String getPaymentId() {
        return paymentId;
    }

    public String getEnrollmentId() {
        return enrollmentId;
```

```
    }

    public double getAmount() {
        return amount;
    }

    public String getPaymentDate() {
        return paymentDate;
    }

    public String getPaymentMethod() {
        return paymentMethod;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public void displayPayment() {
        System.out.println("Payment ID: " + paymentId);
        System.out.println("Enrollment ID: " + enrollmentId);
        System.out.println("Amount: " + amount);
        System.out.println("Payment Date: " + paymentDate);
        System.out.println("Payment Method: " + paymentMethod);
        System.out.println("Payment Status: " + status);
    }
}
```

**Receipt.java:**

```
package model;

public class Receipt {
    private String receiptId;
    private Payment payment;
    private String studentName;
    private String courseName;

    public Receipt(String receiptId, Payment payment,
                   String studentName, String courseName) {
        this.receiptId = receiptId;
        this.payment = payment;
        this.studentName = studentName;
        this.courseName = courseName;
    }
```

```
    public String getReceiptId() {
        return receiptId;
    }

    public Payment getPayment() {
        return payment;
    }

    public void displayReceipt() {
        System.out.println("\n========== PAYMENT RECEIPT ==========");
        System.out.println("Receipt ID: " + receiptId);
        System.out.println("Student Name: " + studentName);
        System.out.println("Course Name: " + courseName);
        System.out.println("Payment ID: " + payment.getPaymentId());
        System.out.println("Amount Paid: " + payment.getAmount());
        System.out.println("Payment Method: " + payment.getPaymentMethod());
        System.out.println("Payment Status: " + payment.getStatus());
        System.out.println("=====================================");
    }
}
```

**EnrollmentService.java:**

```
package service;

import model.Course;
import model.Enrollment;
import model.Student;
import exception.CourseFullException;
import java.util.ArrayList;

public class EnrollmentService {
    private ArrayList<Enrollment> enrollments = new ArrayList<>();

    public boolean enrollStudent(String enrollmentId, Student student,
                                 Course course, String enrollmentDate) {
        try {
            checkCourseCapacity(course);

            Enrollment enrollment = new Enrollment(
                enrollmentId, student, course, enrollmentDate
            );

            enrollments.add(enrollment);
            course.setCapacity(course.getCapacity() - 1);

            System.out.println("Student enrolled successfully!");
            return true;
```

```
        } catch (CourseFullException e) {
            System.out.println(e.getMessage());
            return false;
        }
    }

    private void checkCourseCapacity(Course course)
            throws CourseFullException {
        if (course.getCapacity() <= 0) {
            throw new CourseFullException("Course is full!");
        }
    }

    public void displayAllEnrollments() {
        if (enrollments.isEmpty()) {
            System.out.println("No enrollments found.");
            return;
        }

        for (Enrollment enrollment : enrollments) {
            enrollment.displayEnrollment();
            System.out.println("--------------------");
        }
    }
}
```

**PaymentService.java:**

```
package service;

import model.Payment;
import model.Receipt;
import model.Enrollment;
import java.util.ArrayList;

public class PaymentService {
    private ArrayList<Payment> payments = new ArrayList<>();

    public Receipt processPayment(String paymentId, Enrollment enrollment,
                                  double amount, String paymentDate,
                                  String paymentMethod) {
        if (amount <= 0) {
            System.out.println("Invalid payment amount!");
            return null;
        }

        Payment payment = new Payment(
            paymentId,
```

```
            enrollment.getEnrollmentId(),
            amount,
            paymentDate,
            paymentMethod
        );

        payments.add(payment);

        Receipt receipt = new Receipt(
            "R-" + paymentId,
            payment,
            enrollment.getStudent().getName(),
            enrollment.getCourse().getCourseName()
        );

        System.out.println("Payment processed successfully!");
        return receipt;
    }

    public void displayAllPayments() {
        if (payments.isEmpty()) {
            System.out.println("No payments found.");
            return;
        }

        for (Payment payment : payments) {
            payment.displayPayment();
            System.out.println("---------------------");
        }
    }
}
```

**CourseFullException.java:**

```
package exception;

public class CourseFullException extends Exception {

    public CourseFullException(String message) {
        super(message);
    }
}
```

### 3. Execution Output Screenshot

```
===== ONLINE COURSE ENROLLMENT SYSTEM =====
1. Display Course Details
2. Register Regular Student
3. Register Premium Student
4. Enroll Student
5. View All Enrollments
6. Process Payment
7. View All Payments
8. Exit
Enter your choice: 1
Course ID: C101
Course Name: Java Fundamentals
Fee: 2000.0
Capacity: 2
Instructor: Arun
```

```
===== ONLINE COURSE ENROLLMENT SYSTEM =====
1. Display Course Details
2. Register Regular Student
3. Register Premium Student
4. Enroll Student
5. View All Enrollments
6. Process Payment
7. View All Payments
8. Exit
Enter your choice: 2
Enter student ID: 101
Enter student name: Vinoth P
Enter student email: vinoth@gmail.com
Student registered successfully!
```

```
===== ONLINE COURSE ENROLLMENT SYSTEM =====
1. Display Course Details
2. Register Regular Student
3. Register Premium Student
4. Enroll Student
5. View All Enrollments
6. Process Payment
7. View All Payments
8. Exit
Enter your choice: 4

Registered Students:
101 - Vinoth P
Enter student ID to enroll: 101
Enter enrollment date (DD-MM-YYYY): 12-07-2007
Student enrolled successfully!
```

```
===== ONLINE COURSE ENROLLMENT SYSTEM =====
1. Display Course Details
2. Register Regular Student
3. Register Premium Student
4. Enroll Student
5. View All Enrollments
6. Process Payment
7. View All Payments
8. Exit
Enter your choice: 5
Enrollment ID: E101
Student: Vinoth P
Course: Java Fundamentals
Enrollment Date: 12-07-2007
Status: Pending Payment
--------------------
```

```
===== ONLINE COURSE ENROLLMENT SYSTEM =====
1. Display Course Details
2. Register Regular Student
3. Register Premium Student
4. Enroll Student
5. View All Enrollments
6. Process Payment
7. View All Payments
8. Exit
Enter your choice: 6

Enrollments:
E101 - Vinoth P - Java Fundamentals
Enter enrollment ID: E101
Enter payment method: UPI
Payment processed successfully!
```

```
===== ONLINE COURSE ENROLLMENT SYSTEM =====
1. Display Course Details
2. Register Regular Student
3. Register Premium Student
4. Enroll Student
5. View All Enrollments
6. Process Payment
7. View All Payments
8. Exit
Enter your choice: 7
Payment ID: P101
Enrollment ID: E101
Amount: 2000.0
Payment Date: 24-10-2026
Payment Method: UPI
Payment Status: Successful
--------------------
```

**4. GitHub Repository Link**

**https://github.com/Samkarpagam07/Java-Assignment/upload/main**